SOUTHEAST UNIVERSITY

Meeting the Challenges of Time

Project on

Inventory Management System

Course name: Database Design Lab **Course Code:** CSE384

Section: 7

Submitted By

Name	ID
Md.Yeasin Arafat	2023100000351
Md.Mahfujur Rahman Lijon	2023100000153
Nazmul Hasan	2023100000130

Submitted To

S I M Adnan
Lecturer, Department of CSE, Southeast University
Initial: IN

Submission Date: 16 - 01 - 2025

Introduction:

An Inventory Management System is a database-based system that is applied to record, organize, and manage information pertaining to products, customers, suppliers and sales transactions. The system aids an organization in monitoring the availability of products, customer orders and suppliers efficiently and in an orderly manner.

Data duplication, lack of data consistency and inability to monitor the level of stock are usually challenges in manual inventory systems. These problems are addressed by using a computerized database system through centralized and organized storage of data.

This project uses a relational database based Inventory Management System. The system keeps the records of the customers, suppliers, product categories, products, orders and order details. The database is developed in accordance with normalization principles to guarantee the data consistency, minimize redundancy and enhancement of the data integrity.

Objectives:

The key goals of this project are:

- To create and develop a database about an inventory management system using a relational database.
- To implement database normalization to Third Normal Form (3NF) to reduce redundancy of data.
- To create appropriate relationships with the help of primary keys and foreign keys.
- To record and administer data pertaining to customers, products, suppliers and orders.
- To conduct different SQL queries to retrieve, analyze and report data.

<u>Requirements Description:</u>

- **The System Narrative Description:**

  The Inventory Management System is designed to manage and organize information related to products, customers, suppliers, and sales orders within a business environment. The system is built on the principle of the relational database model and has a number of interconnected entities that collaborate to facilitate inventory and order management functions.

- **Entities Involved:** The following are the key entities of the system:
  - **Customer:** Stores details about customers like customer ID, name, contact number and city. Products are ordered by the customers.
  - **Supplier:** It includes information about suppliers of products delivered to the business such as the name, contact number and location of the suppliers.
  - **Category:** The different product categories comprise beverage, grocery, electronics, and stationery. All the products fall into one category.
  - **Product:** Stores product related data such as product name, unit price, quantity of stock, and reference to supplier and product category.
  - **Orders:** This document contains customer orders, the date of the order, the means of payment, and the name of the orderer.
  - **OrderItem:** These are items in an order. It retains data concerning products that are part of an order and quantity and selling price. This entity settles the many-to-many association between Orders and Products.

- **Main Functionalities:** The key features of the system are:
  - Keeping customer, supplier records and product categories.
  - Controlling product data, such as price and inventory.
  - Documenting the orders of customers and products in each of their orders.
  - Monitoring sales transactions using order and order item records.

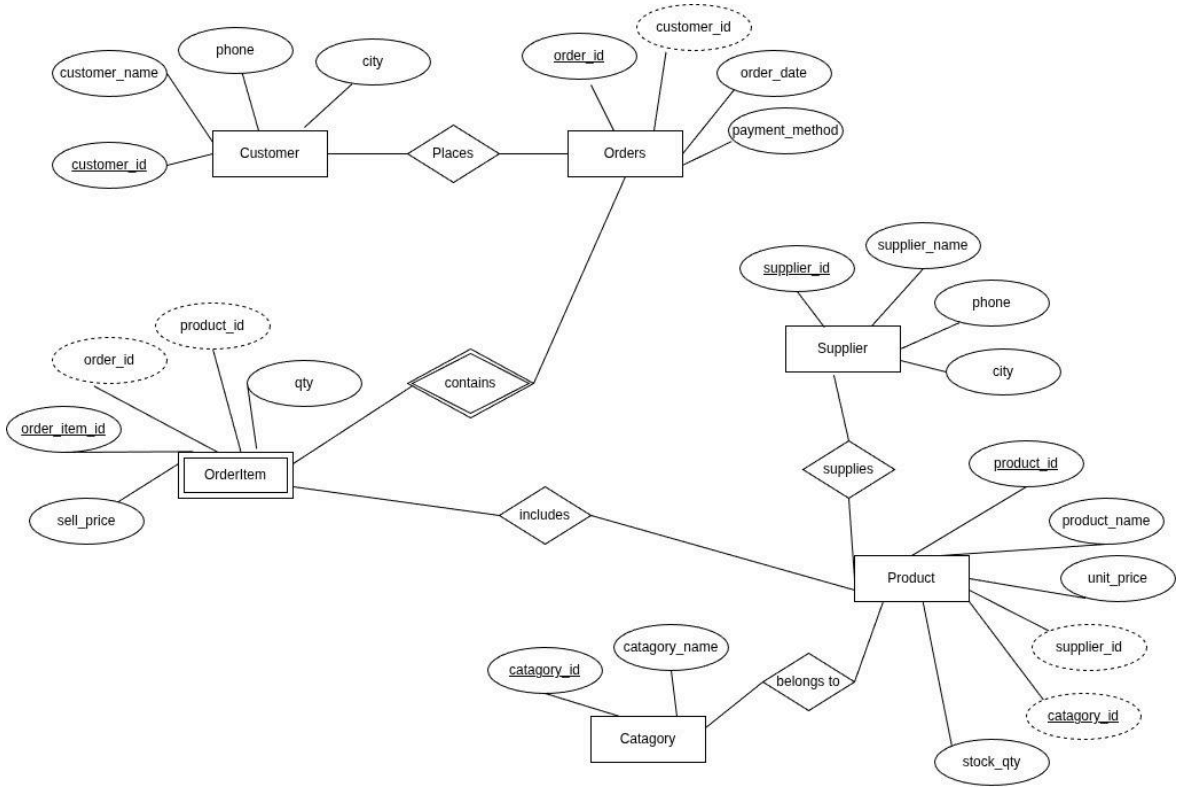
- Searching, filtering, aggregation and combining many tables with SQL queries (doing data retrieval and analysis).

- **Users of the System:** The system will be used by:
  - Store administrators who handle supplier, category and product data.
  - Carter sales personnel or cashiers who note the customer order and order description.
  - Users of the management who examine the inventory and sales data through database queries.

The system is also intended to be used for academic and learning purposes and the system has an appropriate database design, normalization and the implementation of SQL queries.

ER Diagram:

Tool used (draw.io):



Relational Schema:

1. Customer (customer_id INT PRIMARY KEY,customer_name VARCHAR(100),phone VARCHAR(20), city VARCHAR(50)).
2. Supplier (supplier_id PK, supplier_name, phone, city)
3. Category (category_id PK, category_name)
4. Product (product_id PK, product_name, unit_price, stock_qty, supplier_id FK → Supplier(supplier_id), category_id FK → Category(category_id))
5. Orders (order_id PK, order_date, payment_method, customer_id FK → Customer(customer_id))
6. OrderItem (order_id PK, FK → Orders(order_id), product_id PK, FK → Product(product_id), qty, sell_price)

Normalization:

- **1NF:**
  - Each record (row) is unique, which means there are no duplicate rows.
  - All entries in a column must be of the same data type.
  - Each cell must contain only one value (no repeating groups or arrays).

Order ID	Order Date	Customer Nam	Customer Phor	Customer City	Product ID	Product Name	Category Nam	Supplier Name	Qty	Sell Price
101	2026-01-05	Rahim Uddin	1920000001	Dhaka	P01	Coca Cola 500n	Beverage	Metro Beverage	2	40
101	2026-01-05	Rahim Uddin	1920000001	Dhaka	P03	Potato Chips 50	Snack	City Wholesale	3	25
102	2026-01-06	Karim Mia	1920000002	Dhaka	P02	Pepsi 500ml	Beverage	Metro Beverage	4	38
102	2026-01-06	Karim Mia	1920000002	Dhaka	P04	Chocolate Biscu	Snack	Bengal Traders	6	15
103	2026-01-07	Sumaiya Akter	1920000003	Dhaka	P05	Milk 1L	Dairy	Fresh Importers	2	90
104	2026-01-08	Rasel Hossain	1920000004	Chattogram	P08	Power Bank 100	Electronics	Tech Parts BD	1	1200

- **2NF:**
  - It is in 1NF.
  - It has no partial dependencies, meaning that all non-prime attributes are fully functionally dependent on the whole candidate key (not just part of it).
  - Partial dependency occurs when an attribute is dependent on only part of a composite (multi-column) key. A table is in 2NF when all attributes depend on the entire candidate key.

Table 1: Orders

order_id	order_date	customer_name	customer_phone	customer_city
101	2026-01-05	Rahim Uddin	1920000001	Dhaka
102	2026-01-06	Karim Mia	1920000002	Dhaka
103	2026-01-07	Sumaiya Akter	1920000003	Dhaka
104	2026-01-08	Rasel Hossain	1920000004	Chattogram

Table 2: Product

product_id	product_name	category_name	supplier_name
P01	Coca Cola 500m	Beverage	Metro Beverages
P02	Pepsi 500ml	Beverage	Metro Beverages
P03	Potato Chips 50	Snack	City Wholesale
P04	Chocolate Biscu	Snack	Bengal Traders
P05	Milk 1L	Dairy	Fresh Importers
P08	Power Bank 100	Electronics	Tech Parts BD

Table 3: OrderItem

order_id	product_id	qty	sell_price
101	P01	2	40
101	P03	3	25
102	P02	4	38
102	P04	6	15
103	P05	2	90
104	P08	1	1200

- **3NF:**
  - It is in 2NF.
  - It has no transitive dependencies, meaning that non-prime attributes do not depend on other non-prime attributes.
  - A transitive dependency occurs when a non-prime attribute depends on

another non-prime attribute, which in turn depends on the candidate key

Table 1: Customer

customer_id	customer_name	customer_phone	customer_city
C01	Rahim Uddin	1920000001	Dhaka
C02	Karim Mia	1920000002	Dhaka
C03	Sumaiya Akter	1920000003	Dhaka
C04	Rasel Hossain	1920000004	Chattogram

Table 2: Orders

order_id	order_date	customer_id	payment_method
101	2026-01-05	C01	CASH
102	2026-01-06	C02	BKASH
103	2026-01-07	C03	CASH
104	2026-01-08	C04	CARD

Table 3: Supplier

supplier_id	supplier_name
S01	Metro Beverages
S02	City Wholesale
S03	Bengal Traders
S04	Fresh Importers
S05	Tech Parts BD

Table 4: Category

category_id	category_name
CAT01	Beverage
CAT02	Snack
CAT03	Dairy
CAT04	Electronics

Table 5: Product

product_id	product_name	unit_price	stock_qty	supplier_id	category_id
P01	Coca Cola 500m	40	200	S01	CAT01
P02	Pepsi 500ml	38	180	S01	CAT01
P03	Potato Chips 50	25	300	S02	CAT02
P04	Chocolate Biscu	15	350	S03	CAT02
P05	Milk 1L	90	120	S04	CAT03
P08	Power Bank 100	1200	25	S05	CAT04

Table 6: OrderItem

order_id	product_id	qty	sell_price
101	P01	2	40
101	P03	3	25
102	P02	4	38
102	P04	6	15
103	P05	2	90
104	P08	1	1200

Database Schema:

Customer:

Attribute	Data Type	Key	Reference
customer_id	INT	PK	—
customer_name	VARCHAR(100)	—	—
phone	VARCHAR(20)	—	—
city	VARCHAR(50)	—	—

Supplier:

Attribute	Data Type	Key	Reference
supplier_id	INT	PK	—
supplier_name	VARCHAR(100)	—	—
phone	VARCHAR(20)	—	—
city	VARCHAR(50)	—	—

Category:

Attribute	Data Type	Key	Reference
category_id	INT	PK	—
category_name	VARCHAR(50)	—	—

Product:

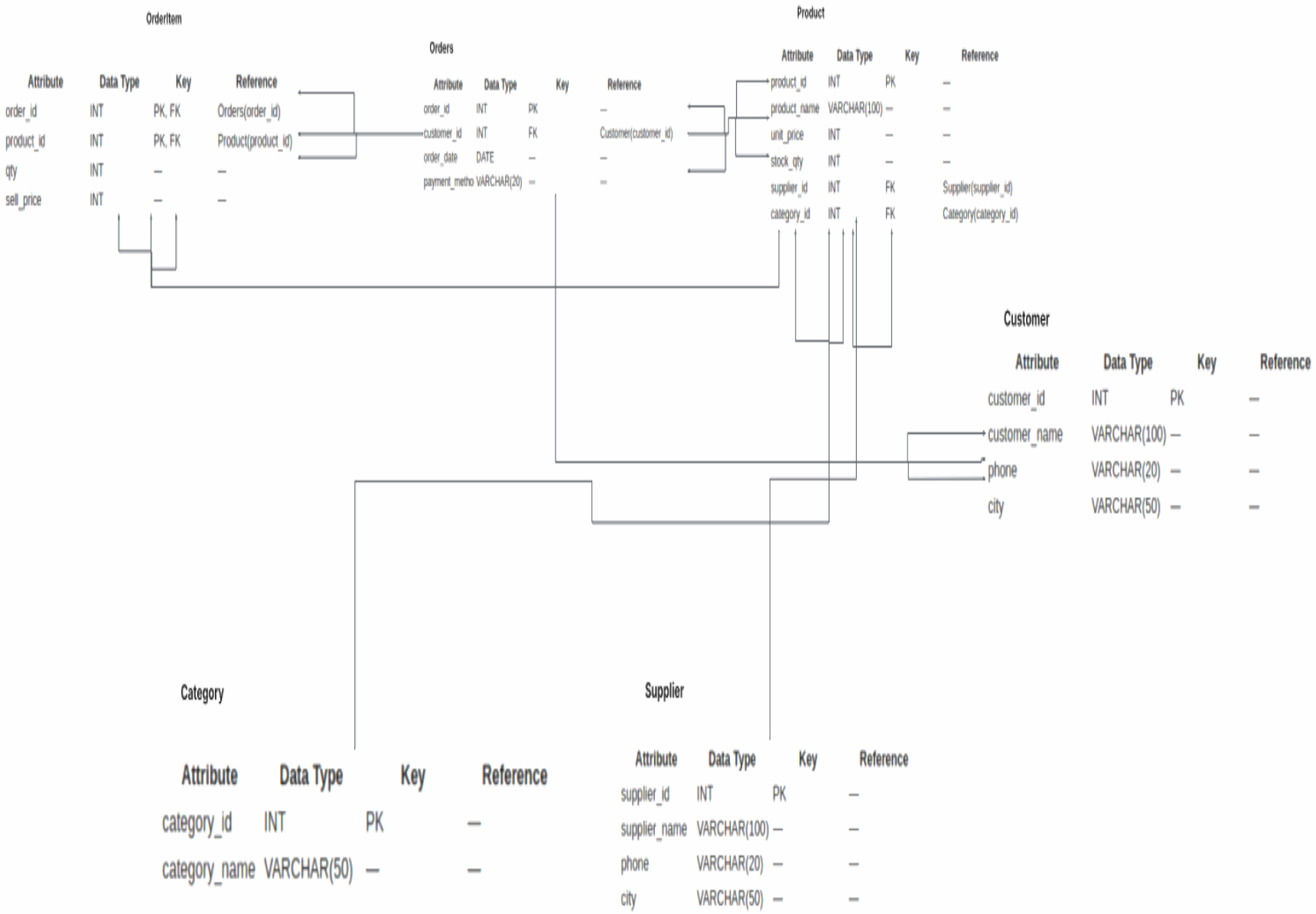
Attribute	Data Type	Key	Reference
product_id	INT	PK	—
product_name	VARCHAR(100)	—	—
unit_price	INT	—	—
stock_qty	INT	—	—
supplier_id	INT	FK	Supplier(supplier_id)
category_id	INT	FK	Category(category_id)

Orders:

Attribute	Data Type	Key	Reference
order_id	INT	PK	—
customer_id	INT	FK	Customer(customer_id)
order_date	DATE	—	—
payment_metho	VARCHAR(20)	—	—

OrderItem:

Attribute	Data Type	Key	Reference
order_id	INT	PK, FK	Orders(order_id)
product_id	INT	PK, FK	Product(product_id)
qty	INT	—	—
sell_price	INT	—	—


OrderItem
Attribute Data Type Key Reference
order_id INT PK, FK Orders(order_id)
product_id INT PK, FK Product(product_id)
qty INT – –
sell_price INT – –
Orders
Attribute Data Type Key Reference
order_id INT PK –
customer_id INT FK Customer(customer_id)
order_date DATE – –
payment_metho VARCHAR(20) – –
Product
Attribute Data Type Key Reference
product_id INT PK –
product_name VARCHAR(100) – –
unit_price INT – –
stock_qty INT – –
supplier_id INT FK Supplier(supplier_id)
category_id INT FK Category(category_id)
Customer
Attribute Data Type Key Reference
customer_id INT PK –
customer_name VARCHAR(100) – –
phone VARCHAR(20) – –
city VARCHAR(50) – –
Category
Attribute Data Type Key Reference
category_id INT PK –
category_name VARCHAR(50) – –
Supplier
Attribute Data Type Key Reference
supplier_id INT PK –
supplier_name VARCHAR(100) – –
phone VARCHAR(20) – –
city VARCHAR(50) – –

Database Implementation (SQL Scripts):

1. **Create Database:**
   Query: `create database inventory_management_system;`

   Output:

   MySQL returned an empty result set (i.e. zero rows). (Query took 0.0019 seconds.)

   ```
   CREATE DATABASE inventory_management_system;
   ```

   [Edit inline] [Edit] [Create PHP code]

   Error: #1046 No database selected

2. **Create Table:**

   Query:
   ```
   CREATE TABLE Customer ( customer_id INT PRIMARY KEY, customer_name VARCHAR(100), phone VARCHAR(20), city VARCHAR(50) );
   ```

   Output:

   MySQL returned an empty result set (i.e. zero rows). (Query took 0.0014 seconds.)

   ```
   CREATE TABLE Customer ( customer_id INT PRIMARY KEY, customer_name VARCHAR(100), phone VARCHAR(20),
   city VARCHAR(50) );
   ```

   [Edit inline] [Edit] [Create PHP code]

   Query:
   ```
   CREATE TABLE Supplier ( supplier_id INT PRIMARY KEY, supplier_name VARCHAR(100), phone VARCHAR(20), city VARCHAR(50) );
   ```

   Output:

   MySQL returned an empty result set (i.e. zero rows). (Query took 0.0006 seconds.)

   ```
   CREATE TABLE Supplier ( supplier_id INT PRIMARY KEY, supplier_name VARCHAR(100), phone VARCHAR(20),
   city VARCHAR(50) );
   ```

   [Edit inline] [Edit] [Create PHP code]

Query: `CREATE TABLE Category ( category_id INT PRIMARY KEY, category_name VARCHAR(50) );`

Output:

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0008 seconds.)

```
CREATE TABLE Category ( category_id INT PRIMARY KEY, category_name VARCHAR(50) );
```

[Edit inline] [Edit] [Create PHP code]

Query: `CREATE TABLE Product ( product_id INT PRIMARY KEY, product_name VARCHAR(100), unit_price INT, supplier_id INT, category_id INT, stock_qty INT, FOREIGN KEY (supplier_id) REFERENCES Supplier(supplier_id), FOREIGN KEY (category_id) REFERENCES Category(category_id) );`

Output:

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0005 seconds.)

```
CREATE TABLE Product ( product_id INT PRIMARY KEY, product_name VARCHAR(100), unit_price INT,
supplier_id INT, category_id INT, stock_qty INT, FOREIGN KEY (supplier_id) REFERENCES
Supplier(supplier_id), FOREIGN KEY (category_id) REFERENCES Category(category_id) );
```

[Edit inline] [Edit] [Create PHP code]

Query: `CREATE TABLE Orders ( order_id INT PRIMARY KEY, customer_id INT, order_date DATE, payment_method VARCHAR(20), FOREIGN KEY (customer_id) REFERENCES Customer(customer_id) );`

Output:

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0008 seconds.)

```
CREATE TABLE Orders ( order_id INT PRIMARY KEY, customer_id INT, order_date DATE, payment_method
VARCHAR(20), FOREIGN KEY (customer_id) REFERENCES Customer(customer_id) );
```

[Edit inline] [Edit] [Create PHP code]

Query: `CREATE TABLE OrderItem ( order_item_id INT PRIMARY KEY, order_id INT, product_id INT, qty INT, sell_price INT, FOREIGN KEY (order_id) REFERENCES Orders(order_id), FOREIGN KEY (product_id) REFERENCES Product(product_id) );`

Output:

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0019 seconds.)

```
CREATE TABLE OrderItem ( order_item_id INT PRIMARY KEY, order_id INT, product_id INT, qty INT,
sell_price INT, FOREIGN KEY (order_id) REFERENCES Orders(order_id), FOREIGN KEY (product_id)
REFERENCES Product(product_id) );
```

[Edit inline] [Edit] [Create PHP code]

3. **Insert Table Value:**

Query:

```
INSERT INTO Customer (customer_id, customer_name, phone,
city) VALUES
(1,'Rahim Uddin','01920000001','Dhaka'),
(2,'Karim Mia','01920000002','Dhaka'),
(3,'Sumaiya Akter','01920000003','Dhaka'),
(4,'Rasel Hossain','01920000004','Chattogram'),
(5,'Nusrat Jahan','01920000005','Sylhet'),
(6,'Tanvir Hasan','01920000006','Rajshahi'),
(7,'Mim Islam','01920000007','Khulna'),
(8,'Afsana Tuli','01920000008','Cumilla'),
(9,'Shafin Ahmed','01920000009','Dhaka'),
(10,'Jannat Ara','01920000010','Dhaka');
```

Output:

10 rows inserted. (Query took 0.0023 seconds.)

```
INSERT INTO Customer (customer_id, customer_name, phone, city) VALUES (1,'Rahim
Uddin','01920000001','Dhaka'), (2,'Karim Mia','01920000002','Dhaka'), (3,'Sumaiya
Akter','01920000003','Dhaka'), (4,'Rasel Hossain','01920000004','Chattogram'), (5,'Nusrat
Jahan','01920000005','Sylhet'), (6,'Tanvir Hasan','01920000006','Rajshahi'), (7,'Mim
Islam','01920000007','Khulna'), (8,'Afsana Tuli','01920000008','Cumilla'), (9,'Shafin
Ahmed','01920000009','Dhaka'), (10,'Jannat Ara','01920000010','Dhaka');
```

[Edit inline] [Edit] [Create PHP code]

Query:

```
INSERT INTO Supplier (supplier_id, supplier_name, phone,
city) VALUES
(1,'Bengal Traders','01810000001','Dhaka'),
(2,'City Wholesale','01810000002','Chattogram'),
(3,'Sylhet Supply','01810000003','Sylhet'),
(4,'Rajshahi Mart','01810000004','Rajshahi'),
(5,'Metro Beverages','01810000005','Dhaka'),
(6,'Tech Parts BD','01810000006','Dhaka'),
(7,'HomeCare Distributors','01810000007','Cumilla'),
(8,'Green Agro','01810000008','Bogura'),
(9,'Stationery Hub','01810000009','Dhaka'),
(10,'Fresh Importers','01810000010','Dhaka');
```

Output:

✔ 10 rows inserted. (Query took 0.0023 seconds.)

```
INSERT INTO Supplier (supplier_id, supplier_name, phone, city) VALUES (1,'Bengal
Traders','01810000001','Dhaka'), (2,'City Wholesale','01810000002','Chattogram'), (3,'Sylhet
Supply','01810000003','Sylhet'), (4,'Rajshahi Mart','01810000004','Rajshahi'), (5,'Metro
Beverages','01810000005','Dhaka'), (6,'Tech Parts BD','01810000006','Dhaka'), (7,'HomeCare
Distributors','01810000007','Cumilla'), (8,'Green Agro','01810000008','Bogura'), (9,'Stationery
Hub','01810000009','Dhaka'), (10,'Fresh Importers','01810000010','Dhaka');
```

[Edit inline] [Edit] [Create PHP code]

Query:

```
INSERT INTO Category (category_id, category_name) VALUES
(1,'Beverage'),
(2,'Snack'),
(3,'Dairy'),
(4,'Electronics'),
(5,'Stationery'),
(6,'Cleaning'),
(7,'Grocery'),
(8,'Personal Care'),
(9,'Baby Care'),
(10,'Frozen');
```

Output:

10 rows inserted. (Query took 0.0012 seconds.)

```
INSERT INTO Category (category_id, category_name) VALUES (1,'Beverage'), (2,'Snack'), (3,'Dairy'),
(4,'Electronics'), (5,'Stationery'), (6,'Cleaning'), (7,'Grocery'), (8,'Personal Care'), (9,'Baby
Care'), (10,'Frozen');
```

[Edit inline] [Edit] [Create PHP code]

Query:

```
INSERT INTO Product (product_id, product_name,
unit_price, supplier_id, category_id, stock_qty) VALUES
(101,'Coca Cola 500ml',40,5,1,200),
(102,'Pepsi 500ml',38,5,1,180),
(103,'Potato Chips 50g',25,2,2,300),
(104,'Chocolate Biscuit',15,1,2,350),
(105,'Milk 1L',90,10,3,120),
(106,'Butter 200g',120,10,3,80),
(107,'USB Cable',150,6,4,60),
(108,'Power Bank 10000mAh',1200,6,4,25),
(109,'Notebook A4',60,9,5,400),
(110,'Ball Pen',10,9,5,1000);
```

Output:

10 rows inserted. (Query took 0.0029 seconds.)

```
INSERT INTO Product (product_id, product_name, unit_price, supplier_id, category_id, stock_qty)
VALUES (101,'Coca Cola 500ml',40,5,1,200), (102,'Pepsi 500ml',38,5,1,180), (103,'Potato Chips
50g',25,2,2,300), (104,'Chocolate Biscuit',15,1,2,350), (105,'Milk 1L',90,10,3,120), (106,'Butter
200g',120,10,3,80), (107,'USB Cable',150,6,4,60), (108,'Power Bank 10000mAh',1200,6,4,25),
(109,'Notebook A4',60,9,5,400), (110,'Ball Pen',10,9,5,1000);
```

[Edit inline] [Edit] [Create PHP code]

Query:

```
INSERT INTO Orders (order_id, customer_id, order_date,
payment_method) VALUES
(1,1,'2026-01-05','CASH'),
(2,2,'2026-01-06','BKASH'),
(3,3,'2026-01-06','CASH'),
(4,4,'2026-01-07','CARD'),
(5,5,'2026-01-07','CASH'),
(6,6,'2026-01-08','BKASH'),
(7,7,'2026-01-09','CASH'),
```

```
(8,8,'2026-01-10','CASH'),
(9,9,'2026-01-11','CARD'),
(10,10,'2026-01-12','CASH');
```

Output:

✔ 10 rows inserted. (Query took 0.0024 seconds.)

```
INSERT INTO Orders (order_id, customer_id, order_date, payment_method) VALUES (1,1,'2026-01-05','CASH'), (2,2,'2026-01-06','BKASH'), (3,3,'2026-01-06','CASH'), (4,4,'2026-01-07','CARD'), (5,5,'2026-01-07','CASH'), (6,6,'2026-01-08','BKASH'), (7,7,'2026-01-09','CASH'), (8,8,'2026-01-10','CASH'), (9,9,'2026-01-11','CARD'), (10,10,'2026-01-12','CASH');
```

[Edit inline] [Edit] [Create PHP code]

Query:

```
INSERT INTO OrderItem (order_item_id, order_id, product_id, qty, sell_price) VALUES
(1,1,101,2,40),(2,1,103,3,25),
(3,1,110,10,10),(4,2,102,4,38),
(5,2,104,6,15),(6,3,105,2,90),
(7,3,109,5,60),(8,4,108,1,1200),
(9,4,107,2,150),(10,5,105,1,90),
(11,5,103,4,25),(12,6,110,20,10),
(13,6,104,5,15),(14,7,101,3,40),
(15,7,106,1,120),(16,8,109,10,60),
(17,8,110,30,10),(18,9,102,2,38),
(19,9,103,2,25),(20,10,104,8,15);
```

Output:

✔ 20 rows inserted. (Query took 0.0007 seconds.)

```
INSERT INTO OrderItem (order_item_id, order_id, product_id, qty, sell_price) VALUES
(1,1,101,2,40), (2,1,103,3,25), (3,1,110,10,10), (4,2,102,4,38), (5,2,104,6,15), (6,3,105,2,90),
(7,3,109,5,60), (8,4,108,1,1200), (9,4,107,2,150), (10,5,105,1,90), (11,5,103,4,25),
(12,6,110,20,10), (13,6,104,5,15), (14,7,101,3,40), (15,7,106,1,120), (16,8,109,10,60),
(17,8,110,30,10), (18,9,102,2,38), (19,9,103,2,25), (20,10,104,8,15);
```

[Edit inline] [Edit] [Create PHP code]

SQL Queries:

Basic SELECT + WHERE + ORDER BY + LIKE

Query: `SELECT * FROM Product;`

Output:

Showing rows 0 - 9 (10 total, Query took 0.0007 seconds.)

```
SELECT * FROM Product;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25

Extra options

	product_id	product_name	unit_price	supplier_id	category_id	stock_
Edit Copy Delete	101	Coca Cola 500ml	40	5	1	
Edit Copy Delete	102	Pepsi 500ml	38	5	1	
Edit Copy Delete	103	Potato Chips 50g	25	2	2	
Edit Copy Delete	104	Chocolate Biscuit	15	1	2	
Edit Copy Delete	105	Milk 1L	90	10	3	
Edit Copy Delete	106	Butter 200g	120	10	3	
Edit Copy Delete	107	USB Cable	150	6	4	
Edit Copy Delete	108	Power Bank 10000mAh	1200	6	4	
Edit Copy Delete	109	Notebook A4	60	9	5	
Edit Copy Delete	110	Ball Pen	10	9	5	

Query: `SELECT product_name, unit_price FROM Product WHERE unit_price > 100;`

Output:

Showing rows 0 - 2 (3 total, Query took 0.0005 seconds.)

```
SELECT product_name, unit_price FROM Product WHERE unit_price > 100;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25

Extra options

	product_name	unit_price
Edit Copy Delete	Butter 200g	120
Edit Copy Delete	USB Cable	150
Edit Copy Delete	Power Bank 10000mAh	1200

Check all *With selected:* Edit Copy Delete Export

Query: `SELECT product_name FROM Product WHERE product_name LIKE 'P%';`

Output:

Showing rows 0 - 2 (3 total, Query took 0.0009 seconds.)

```
SELECT product_name FROM Product WHERE product_name LIKE 'P%';
```

Profiling [Edit inline] [Edit] [Exp

Show all | Number of rows: 25

Extra options

←T→			product_name
Edit	Copy	Delete	Pepsi 500ml
Edit	Copy	Delete	Potato Chips 50g
Edit	Copy	Delete	Power Bank 10000mAh

Query: `SELECT * FROM Customer ORDER BY customer_name ASC;`

Output:

Showing rows 0 - 9 (10 total, Query took 0.0011 seconds.) [customer_name: **AFSANA TULI... - TANVIR**

```
SELECT * FROM Customer ORDER BY customer_name ASC;
```

Profiling [Edit inline] [Edit] [Explain SQL] [

Show all | Number of rows: 25 Filter rows: Search this table Sort by key:

Extra options

←T→			customer_id	customer_name 1	phone	city
Edit	Copy	Delete	8	Afsana Tuli	01920000008	Cumilla
Edit	Copy	Delete	10	Jannat Ara	01920000010	Dhaka
Edit	Copy	Delete	2	Karim Mia	01920000002	Dhaka
Edit	Copy	Delete	7	Mim Islam	01920000007	Khulna
Edit	Copy	Delete	5	Nusrat Jahan	01920000005	Sylhet
Edit	Copy	Delete	1	Rahim Uddin	01920000001	Dhaka
Edit	Copy	Delete	4	Rasel Hossain	01920000004	Chattogram
Edit	Copy	Delete	9	Shafin Ahmed	01920000009	Dhaka
Edit	Copy	Delete	3	Sumaiya Akter	01920000003	Dhaka
Edit	Copy	Delete	6	Tanvir Hasan	01920000006	Rajshahi

Query: `ELECT * FROM Orders ORDER BY order_date DESC;`

Output:

Showing rows 0 - 9 (10 total, Query took 0.0005 seconds.) [order_date: **2026-01-12... - 2026-01-05...**]

```
SELECT * FROM Orders ORDER BY order_date DESC;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Cr

Show all | Number of rows: 25 Filter rows: Search this table Sort by key:

Extra options

←T→	order_id	customer_id	order_date 1	payment_method
Edit Copy Delete	10	10	2026-01-12	CASH
Edit Copy Delete	9	9	2026-01-11	CARD
Edit Copy Delete	8	8	2026-01-10	CASH
Edit Copy Delete	7	7	2026-01-09	CASH
Edit Copy Delete	6	6	2026-01-08	BKASH
Edit Copy Delete	5	5	2026-01-07	CASH
Edit Copy Delete	4	4	2026-01-07	CARD
Edit Copy Delete	3	3	2026-01-06	CASH
Edit Copy Delete	2	2	2026-01-06	BKASH
Edit Copy Delete	1	1	2026-01-05	CASH

-- IN / NOT IN / BETWEEN / NOT BETWEEN / IS NULL / LIMIT

Query: `SELECT product_name, unit_price FROM Product WHERE unit_price BETWEEN 20 AND 100;`

Output:

Showing rows 0 - 4 (5 total, Query took 0.0016 seconds.)

```
SELECT product_name, unit_price FROM Product WHERE unit_price BETWEEN 20 AND 100;
```

Profiling [Edit inline] [Edit] [Explain SQ

Show all | Number of rows: 25 Filter rows: Search this table Sort by

Extra options

←T→	product_name	unit_price
Edit Copy Delete	Coca Cola 500ml	40
Edit Copy Delete	Pepsi 500ml	38
Edit Copy Delete	Potato Chips 50g	25
Edit Copy Delete	Milk 1L	90
Edit Copy Delete	Notebook A4	60

Query: `SELECT product_name FROM Product WHERE unit_price NOT BETWEEN 20 AND 100;`

Output:

Showing rows 0 - 4 (5 total, Query took 0.0005 seconds.)

```
SELECT product_name FROM Product WHERE unit_price NOT BETWEEN 20 AND 100;
```

Profiling [Edit inline] [Edit] [

Show all | Number of rows: 25 Filter rows: Search this table

Extra options

←T→			product_name
Edit	Copy	Delete	Chocolate Biscuit
Edit	Copy	Delete	Butter 200g
Edit	Copy	Delete	USB Cable
Edit	Copy	Delete	Power Bank 10000mAh
Edit	Copy	Delete	Ball Pen

Query: `SELECT product_name FROM Product WHERE category_id IN (1,2);`

Output:

Showing rows 0 - 3 (4 total, Query took 0.0007 seconds.)

```
SELECT product_name FROM Product WHERE category_id IN (1,2);
```

Profiling [Ec

Show all | Number of rows: 25 Filter rows: Search this

Extra options

←T→			product_name
Edit	Copy	Delete	Coca Cola 500ml
Edit	Copy	Delete	Pepsi 500ml
Edit	Copy	Delete	Potato Chips 50g
Edit	Copy	Delete	Chocolate Biscuit

Query: `SELECT supplier_name FROM Supplier WHERE city NOT IN ('Dhaka');`

Output:

Showing rows 0 - 4 (5 total, Query took 0.0004 seconds.)

```
SELECT supplier_name FROM Supplier WHERE city NOT IN ('Dhaka');
```

Profiling [Edit inli

Show all | Number of rows: 25 Filter rows: Search this tal

Extra options

←T→			supplier_name
Edit	Copy	Delete	City Wholesale
Edit	Copy	Delete	Sylhet Supply
Edit	Copy	Delete	Rajshahi Mart
Edit	Copy	Delete	HomeCare Distributors
Edit	Copy	Delete	Green Agro

Query: `SELECT product_name, unit_price FROM Product ORDER BY unit_price DESC LIMIT 3;`

Output:

Showing rows 0 - 2 (3 total, Query took 0.0006 seconds.) [unit_price: **1200... - 120...**]

```
SELECT product_name, unit_price FROM Product ORDER BY unit_price DESC LIMIT 3;
```

Profiling [Edit inline] [Edit] [Exp

Extra options

←T→			product_name	unit_price 1
Edit	Copy	Delete	Power Bank 10000mAh	1200
Edit	Copy	Delete	USB Cable	150
Edit	Copy	Delete	Butter 200g	120

-- Aggregates

Query: `SELECT AVG(unit_price) AS avg_price FROM Product;`

Output:

Showing rows 0 - 0 (1 total, Query took 0.0014 seconds.)

```
SELECT AVG(unit_price) AS avg_price FROM Product;
```

Profi

Show all | Number of rows: 25 Filter rows: Sea

Extra options

avg_price
174.8000

Query: `SELECT SUM(stock_qty) AS total_stock FROM Product;`

Output:

Showing rows 0 - 0 (1 total, Query took 0.0007 seconds.)

```
SELECT SUM(stock_qty) AS total_stock FROM Product;
```

P

Show all | Number of rows: 25 Filter rows: S

Extra options

total_stock
2715

Query: `SELECT MIN(unit_price) AS min_price, MAX(unit_price) AS max_price FROM Product;`

Output:

Showing rows 0 - 0 (1 total, Query took 0.0009 seconds.)

```
SELECT MIN(unit_price) AS min_price, MAX(unit_price) AS max_price FROM Product;
```

Profiling [Edit inline] [Edit] [Expla

Show all | Number of rows: 25 Filter rows: Search this table

Extra options

min_price	max_price
10	1200

Query:`SELECT COUNT(product_id) AS total_products FROM Product;`

Output:

Your SQL query has been executed successfully.

```
SELECT COUNT(product_id) AS total_products FROM Product;
```

Profi

Extra options

total_products
10

-- GROUP BY + HAVING

Query: `SELECT category_id, COUNT(product_id) AS product_count FROM Product GROUP BY category_id;`

Output:

Showing rows 0 - 4 (5 total, Query took 0.0007 seconds.)

```
SELECT category_id, COUNT(product_id) AS product_count FROM Product GROUP BY category_id;
```

Profiling [Edit inline] [Edit] [Explain SQL] [C

Show all | Number of rows: 25 Filter rows: Search this table

Extra options

category_id	product_count
1	2
2	2
3	2
4	2
5	2

Query:`SELECT supplier_id, SUM(stock_qty) AS supplier_stock FROM Product GROUP BY supplier_id HAVING SUM(stock_qty) >= 200;`

Output:

Showing rows 0 - 4 (5 total, Query took 0.0028 seconds.)

```
SELECT supplier_id, SUM(stock_qty) AS supplier_stock FROM Product GROUP BY supplier_id HAVING SUM(stock_qty)
>= 200;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table

Extra options

				supplier_id	supplier_stock
Edit	Copy	Delete		1	350
Edit	Copy	Delete		2	300
Edit	Copy	Delete		5	380
Edit	Copy	Delete		9	1400
Edit	Copy	Delete		10	200

-- Subquery

Query: `SELECT product_name, unit_price FROM Product WHERE unit_price > (SELECT AVG(unit_price) FROM Product);`

Output:

Showing rows 0 - 0 (1 total, Query took 0.0041 seconds.)

```
SELECT product_name, unit_price FROM Product WHERE unit_price > (SELECT AVG(unit_price) FROM
Product);
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25

Extra options

				product_name	unit_price
Edit	Copy	Delete		Power Bank 10000mAh	1200

-- JOINs

Query: `SELECT p.product_name, c.category_name FROM Product p INNER JOIN Category c ON p.category_id = c.category_id;`

Output:

Showing rows 0 - 9 (10 total, Query took 0.0019 seconds.)

```
SELECT p.product_name, c.category_name FROM Product p INNER JOIN Category c ON p.category_id =
c.category_id;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refres

Show all | Number of rows: 25

Extra options

product_name	category_name
Coca Cola 500ml	Beverage
Pepsi 500ml	Beverage
Potato Chips 50g	Snack
Chocolate Biscuit	Snack
Milk 1L	Dairy
Butter 200g	Dairy
USB Cable	Electronics
Power Bank 10000mAh	Electronics
Notebook A4	Stationery
Ball Pen	Stationery

Query:

```
SELECT o.order_id, o.order_date, cu.customer_name FROM Orders o INNER JOIN Customer cu ON o.customer_id =
cu.customer_id;
```

Output:

Showing rows 0 - 9 (10 total, Query took 0.0007 seconds.)

```
SELECT o.order_id, o.order_date, cu.customer_name FROM Orders o INNER JOIN Customer cu ON
o.customer_id = cu.customer_id;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25

Extra options

order_id	order_date	customer_name
1	2026-01-05	Rahim Uddin
2	2026-01-06	Karim Mia
3	2026-01-06	Sumaiya Akter
4	2026-01-07	Rasel Hossain
5	2026-01-07	Nusrat Jahan
6	2026-01-08	Tanvir Hasan
7	2026-01-09	Mim Islam
8	2026-01-10	Afsana Tuli
9	2026-01-11	Shafin Ahmed
10	2026-01-12	Jannat Ara

Query: `SELECT o.order_id, p.product_name, oi.qty, oi.sell_price FROM OrderItem oi INNER JOIN Orders o ON oi.order_id = o.order_id INNER JOIN Product p ON oi.product_id = p.product_id;`

Output:

Showing rows 0 - 19 (20 total, Query took 0.0013 seconds.)

```
SELECT o.order_id, p.product_name, oi.qty, oi.sell_price FROM OrderItem oi INNER JOIN Orders o ON
oi.order_id = o.order_id INNER JOIN Product p ON oi.product_id = p.product_id;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25

Extra options

order_id	product_name	qty	sell_price
1	Coca Cola 500ml	2	40
1	Potato Chips 50g	3	25
1	Ball Pen	10	10
2	Pepsi 500ml	4	38
2	Chocolate Biscuit	6	15
3	Milk 1L	2	90
3	Notebook A4	5	60
4	Power Bank 10000mAh	1	1200
4	USB Cable	2	150
5	Milk 1L	1	90
5	Potato Chips 50g	4	25
6	Ball Pen	20	10
6	Chocolate Biscuit	5	15
7	Coca Cola 500ml	3	40
7	Butter 200g	1	120
8	Notebook A4	10	60
8	Ball Pen	30	10
9	Pepsi 500ml	2	38
9	Potato Chips 50g	2	25
10	Chocolate Biscuit	8	15

Conclusion and Future Advancement

Conclusion

This project provided practical experience in designing and implementing an Inventory Management System using a relational database approach. Through this project, important concepts of database design such as entity–relationship modeling, primary and foreign keys, normalization up to Third Normal Form (3NF), and relational schema

conversion were learned and applied. The project helped in understanding how real-world data related to customers, products, suppliers, and orders can be structured efficiently to reduce redundancy and maintain data integrity.

During the design phase, identifying correct entities, relationships, and keys required careful analysis to ensure proper normalization. Challenges were faced in handling many-to-many relationships and weak entities, which were resolved by introducing an associative table and identifying relationships correctly. During implementation, maintaining consistency between the ER diagram, relational schema, and SQL table structure required attention to detail. Writing SQL queries also improved understanding of data retrieval using filtering, aggregation, and join operations.

Future Advancement

The current system is limited to database design and basic data manipulation for academic purposes. In the future, the system can be enhanced by developing a user-friendly graphical interface for data entry and reporting. Additional features such as automatic stock update, user roles and authentication, and invoice generation can be implemented. The database can also be extended to support real-time inventory tracking, supplier performance analysis, and sales reporting dashboards. Implementing advanced security measures, backup mechanisms, and scalability support would further improve the system for real-world business use.

Contribution of each team member:

Members	Contribution
MD.Mahfujur Rahman Lijon	**1 - 4**
MD.Yeasin Arafat	**5, 6, 10**
Nazmul Hasan	**7, 8, 9, 11**